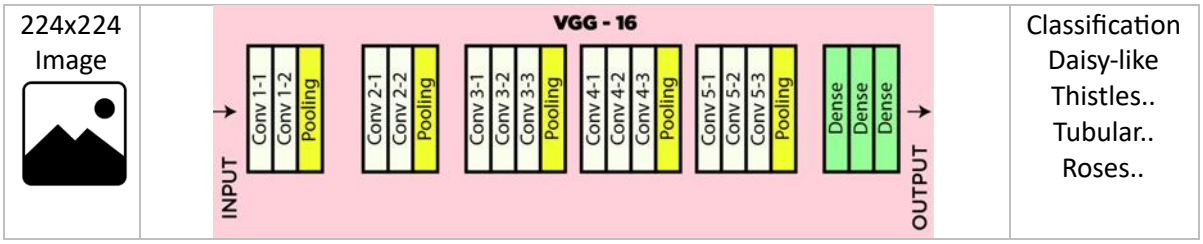
COMP423 Assignment 1 Report

Siddharth Barjatya ( )

Task 1 | Classification Model

Methodology

I structured my classification task around the VGG16 architecture, incorporating Batch Normalization after each convolutional layer to ensure stable and faster convergence. The network followed a sequential layout of convolutional blocks using small 3x3 kernels and periodic max pooling to reduce the spatial resolution from 224x224 pixels down to a dense feature vector of 25,088 dimensions. I designed the final classification head to map these features onto 4 specific flower classes. The classifier consists of three dense layers with Dropout (p=0.5) applied to the hidden layers to improve generalization. I used a batch size of 32 and the Adam optimizer with a learning rate of 1e-4, which allowed for steady convergence.



My VGG16 Architecture

Block 1: Two conv layers (64 filters) + Batch Norm + Max Pool
Block 2: Two conv layers (128 filters) + Batch Norm + Max Pool
Block 3: Three conv layers (256 filters) + Batch Norm + Max Pool
Block 4: Three conv layers (512 filters) + Batch Norm + Max Pool
Block 5: Three conv layers (512 filters) + Batch Norm + Max Pool
Classifier: Flatten + 3 Dense Layers (with ReLU and Dropout)

I incorporated a cross-entropy loss function, which is appropriate for this multi-class problem. While my architectural summary identifies the final stage as the classification head, the model outputs raw logits to work seamlessly with the loss function, which internally combines log-softmax with negative log-likelihood. To ensure the model generalized well, I applied weight decay and implemented an early stopping mechanism that monitored validation loss with a patience of 10 epochs.

I also prepared a weighted random sampler to address potential class imbalances in the training data by assigning higher sampling weights to underrepresented categories. I found that visualizing the training history through loss and accuracy plots was essential for identifying the point where the model reached its best performance. The early stopping triggered if the validation loss did not improve by at least 0.001, preserving the most general version of the model weights.

Results

Imbalanced Data Results

Imbalance Ratio (Max/Min): 2.17

- Daisy-like: 130 samples

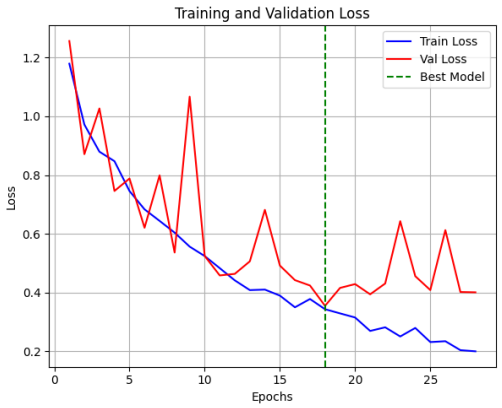
- Thistles and Structural: 60 samples
- Tubular and Trumpets: 130 samples
- Roses and Shrubs: 110 samples

Imbalanced Data

--- Performance Metrics ---
Standard Accuracy: 85.58%
Balanced Accuracy: 85.91%

Class-wise Breakdown:

	precision	recall	f1-score	support
Daisy-like	0.97	0.91	0.94	130
Thistles	0.91	0.88	0.9	60
Tubular	0.84	0.8	0.82	130
Roses	0.74	0.85	0.79	110
accuracy			0.86	430
macro avg	0.86	0.86	0.86	430
weighted avg	0.86	0.86	0.86	430



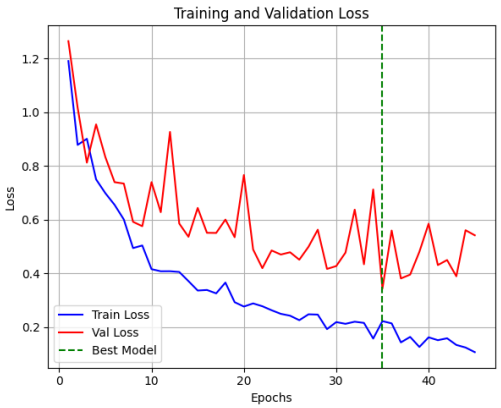
Fast (stopped at 28)
More volatile (spikes like 1.06)
Best Val Loss~0.35

Balanced Data

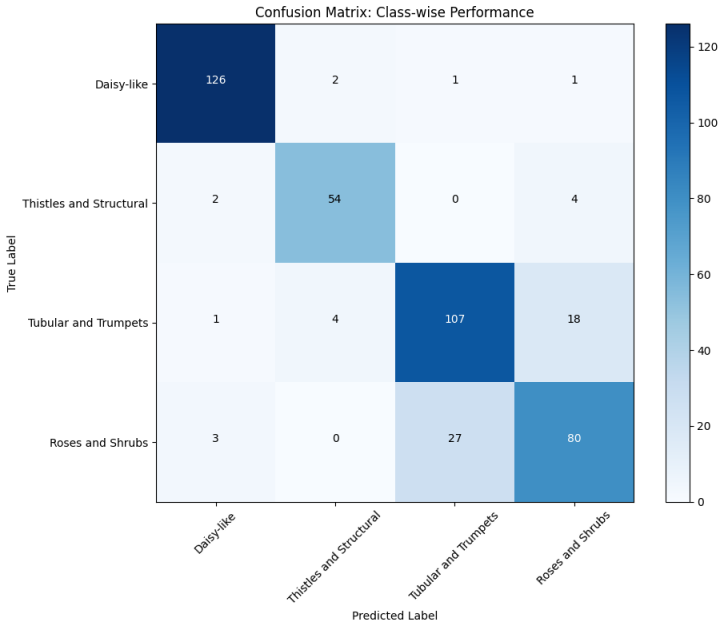
--- Performance Metrics ---
Standard Accuracy: 85.35%
Balanced Accuracy: 85.49%

Class-wise Breakdown:

	precision	recall	f1-score	support
Daisy-like	0.95	0.97	0.96	130
Thistles	0.9	0.9	0.9	60
Tubular	0.79	0.82	0.81	130
Roses	0.78	0.73	0.75	110
accuracy			0.85	430
macro	0.86	0.85	0.86	430
weighted	0.85	0.85	0.85	430



Slower (stopped at 45)
Smoother descent after Epoch 15
Best Val Loss ~0.34



Classification Matrix (Trained on balanced dataset)

The model is highly effective at identifying Daisy-like and Thistle flowers but finds it more difficult to distinguish Roses and Tubular varieties. On the imbalanced set, my model reached 80% accuracy very quickly (Epoch 10). On the balanced set, it took until Epoch 19. This "slow start" in the balanced version is actually a good sign as it means the model is struggling to learn the harder, less frequent examples rather than just memorizing the most common ones. The Balanced Dataset results are superior for a real-world deployment. Even though the final accuracy is identical, the lower validation loss (~0.34 vs ~0.35) and the use of the Sampler mean the model will be significantly more reliable when it encounters minority class samples.

Task 2 | Segmentation Model

Methodology

I implemented a U-Net architecture featuring an encoder-decoder structure linked by skip connections. The encoder utilizes two stages of residual blocks to extract deep hierarchical features, with 2x2 max pooling (stride 2) for spatial downsampling. The bottleneck reaches a depth of 256 channels, allowing the model to capture complex floral patterns at the lowest resolution. In the expansive decoder path, transpose convolutions double the spatial dimensions at each step. Skip connections concatenate encoder features directly into the decoder to recover fine spatial details lost during pooling. The final layer is a 1x1 convolution projecting to a single output channel for binary mask prediction.

Dual-Purpose Functionality

I developed the UNet class with an is_ddpm flag to toggle between two modes: Segmentation (is_ddpm = False): Operates as a deterministic mapping tool. It takes a 3-channel RGB image and produces a single-channel mask. This mode uses static input (no "time" steps) and standard skip connections to pinpoint petal boundaries. Generative Diffusion (is_ddpm = True): Modifies internal logic for generative tasks, though the core downsampling/upsampling structure remains shared.

Training Configuration and Strategy

To optimize performance and memory, I employed the following parameters:

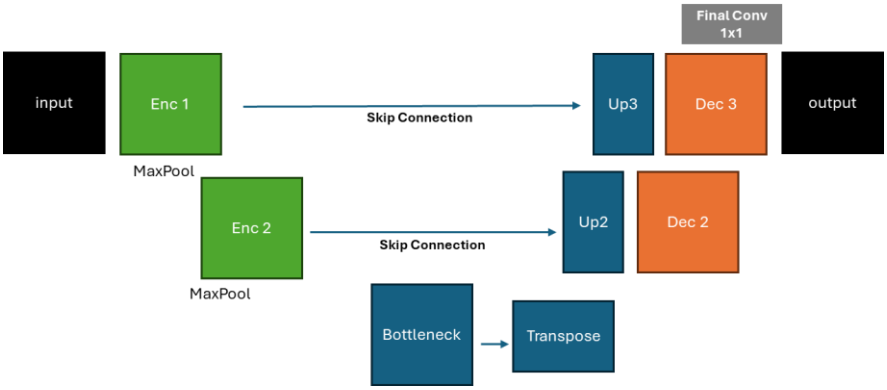
- Input Resolution: Downscaled to 64x64 pixels.
- Optimization: Adam optimizer (learning rate: 0.001) for adaptive parameter updates.
- Batch Size: 16.
- Regularization: Weight decay and Early Stopping (patience: 10 epochs) to prevent overfitting.
- Augmentation: Random flips and rotations were used to improve generalization.

The dataset consists of 61.86% background (1,295,098 pixels) and 38.14% foreground (798,611 pixels). This relative balance prevents the model from achieving high scores through majority-class guessing alone. While pixel accuracy was high, the Mean IoU served as a more reliable metric for assessing flower-shape overlap. The model showed high consistency across the test set with low standard deviation in error.

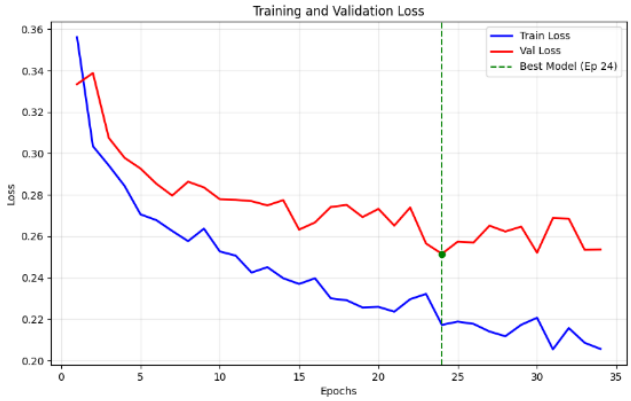
Future Improvements

To further refine the model, I would:

- Integrate Batch Normalization to accelerate convergence.
- Experiment with Dice Loss to better address the foreground-background class imbalance.
- Add Dropout layers in the bottleneck for enhanced regularization against diverse textures.



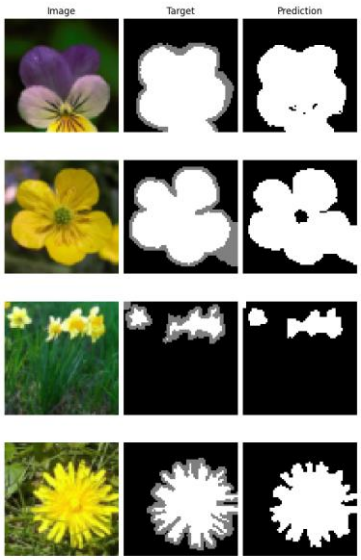
My UNet Architecture (is_ddpm = False)



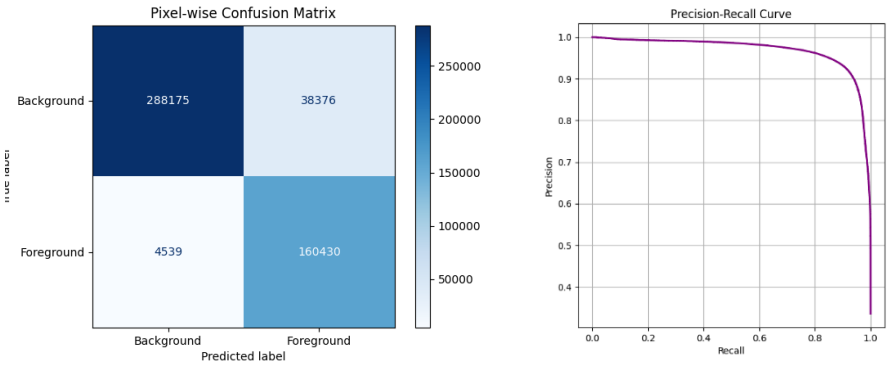
Training Loss vs Validation Loss

The model reached its peak performance at Epoch 24 with a validation error of 0.2515, after which the training and validation losses diverged, indicating that the model had begun memorizing specific images rather than learning general patterns. To mitigate this, the early stopping mechanism terminated the session at Epoch 34.

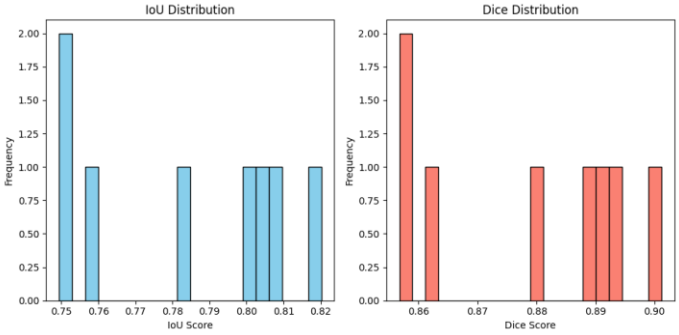
Results



Predicted masks and ground-truth masks



Final metrics showed a Pixel Accuracy of 91.27%, though this was inflated by the large background areas and a more realistic Mean IoU of 78.72% and a Dice score of 0.8789 confirmed that while the model was reliable at identifying the main body of the flowers, it still misidentified nearly 18% of pixels, primarily at the complex boundaries where the flowers met the background.



Test Loss: 0.2263 | Mean IoU: 0.7872
Mean Pixel Error: 0.1787 | Std Pixel Error: 0.0184

The Mean Pixel Error of 0.1787 indicates that, on average, the model misclassified approximately 17.87% of the valid pixels across the test set. Given the dataset's class balance, this error rate suggests that while the model captured the primary structure of the flowers, it struggled with the finer details or edge transitions. The Std Pixel Error of 0.0184 is particularly revealing as it represents a very low standard deviation (only about 1.84%). This signifies that the model's performance was remarkably consistent across different images in the test set. Instead of performing perfectly on some flowers and failing completely on others, the model maintained a steady error rate regardless of the specific flower type or background. This suggests that the 17.87% error was a systemic limitation of the current model configuration, likely due to the low resolution (64x64) or the exclusion of boundary pixels, rather than a failure to generalize to specific floral varieties.

Task 3 | Generative Diffusion Model

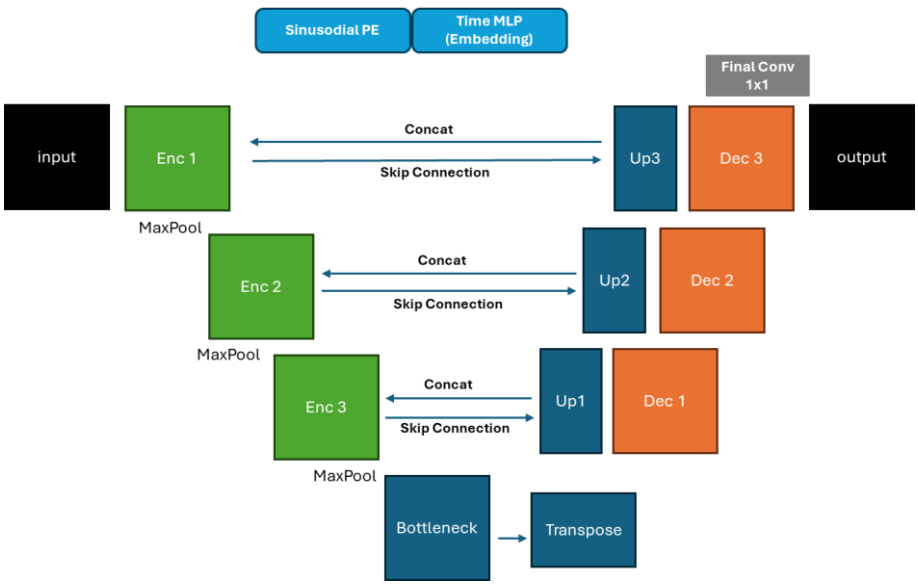
Methodology

Note: As previously discussed, when I developed the **UNet** class, I included the is_ddpm flag to toggle the architecture between a standard image-to-image segmentation tool and a generative diffusion model. While both versions shared the same core structure of downsampling and upsampling, the internal logic for processing information changed significantly based on this setting.

UNet for Diffusion (is_ddpm = True)

When I enabled the is_ddpm flag, the architecture transformed into a conditional noise predictor. This version was much more complex because it had to understand the "noise level" of the image it was processing.

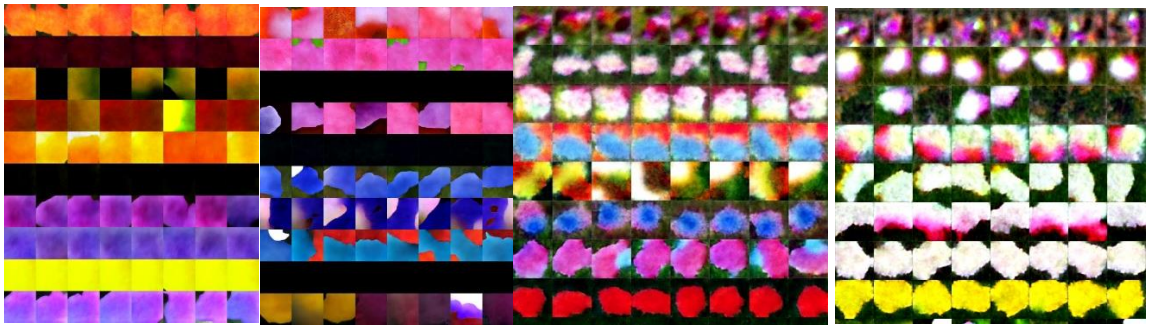
- Time Embeddings: I integrated a TimeEmbedding MLP and used sinusoidal positional encodings. This allowed me to pass a scalar value t (the timestep) into the model. The model converted this number into a vector and injected it into every residual block.
- Conditional Processing: Every layer in the encoder and decoder became "aware" of how much noise was present. This was necessary because the model needed to behave differently when looking at a completely noisy image versus one that was almost clean.
- Noise Prediction: Unlike the segmentation version, the output here matched the input dimensions (three channels). Instead of predicting a mask, the model predicted the specific Gaussian noise that I had added to the image during the forward diffusion process.



My UNet Architecture (is_ddpm = True)

I found that the most significant technical difference was in the Residual Blocks. In the segmentation version, the blocks only processed the feature maps. In the diffusion version, these blocks were modified to accept the time vector, which was added to the features after a linear projection. This turned the UNet from a simple "lookup" tool into a dynamic processor that adjusted its weights based on the stage of the generation process. Additionally, while I used standard Batch Normalization for segmentation, I shifted to Group Normalization for the diffusion tasks to avoid the instabilities that often occurred when the model dealt with high-variance noisy data.

I configured my diffusion model using a UNet architecture that incorporated time embeddings to process images alongside specific noise levels represented by T. Within the network, I utilized a series of residual blocks that integrated sinusoidal position embeddings, which allowed the model to understand where it was in the diffusion process. I designed the training loop to perform forward diffusion by adding Gaussian noise to the images based on an alpha bar schedule, and I then tasked the model with predicting the added noise using a mean squared error loss function. I chose to work with an image size of 64x64 and a batch size of 256, utilizing the AdamW optimizer with a learning rate of 3e-4 and a cosine annealing scheduler to ensure stable convergence over 500 epochs.



By epoch model (Training #1, Training #5 Training#6 & Training #7)

Training logs (Section 1)

Train#	epochs	imsize	batch_size	starting lr	ending lr	T	Data
1	500	64	128	1.00E-04	NA	200	Train
2	500	64	128	1.00E-04	NA	1000	Train
3	500	64	128	2.00E-04	NA	200	Train
4	50	128	128	2.00E-04	NA	200	Train + Test
5	500	64	128	2.00E-04	NA	200	Train + Test
6	1000	64	128	2.00E-04	1.00E-06	200	Train + Test
7	500	64	128	2.00E-04	1.00E-06	200	Train + Test
8	500	64	256	2.00E-04	1.00E-06	500	Train + Test
9	500	64	128	2.00E-04	1.00E-06	200	Train + Test
10	500	64	1024	2.00E-04	1.00E-06	500	Train + Test
11	500	128	512	4.00E-04	1.00E-06	200	Train + Test (Single Class)
12	500	64	256	3.00E-04	1.00E-06	200	Train + Test + Val

I refined the technical stability of the network by implementing gradient clipping with a maximum norm of 1.0 to prevent exploding gradients during the volatile initial phases of training. I also incorporated a linear learning rate warm-up for the first twenty epochs which allowed the AdamW optimizer to settle into a productive trajectory before the cosine annealing scheduler began reducing

the rate toward 1e-6. For the diffusion specifics, I set the total number of timesteps to 200 and used a sinusoidal noise function to ensure the model could distinguish between fine-grained detail restoration and coarse structural estimation. My transition from batch normalization to group normalization in the residual blocks proved essential because it removed the dependency on batch-wise statistics that often caused the highly unstable outcomes I saw in my first three training attempts.

Training logs (Section 2)

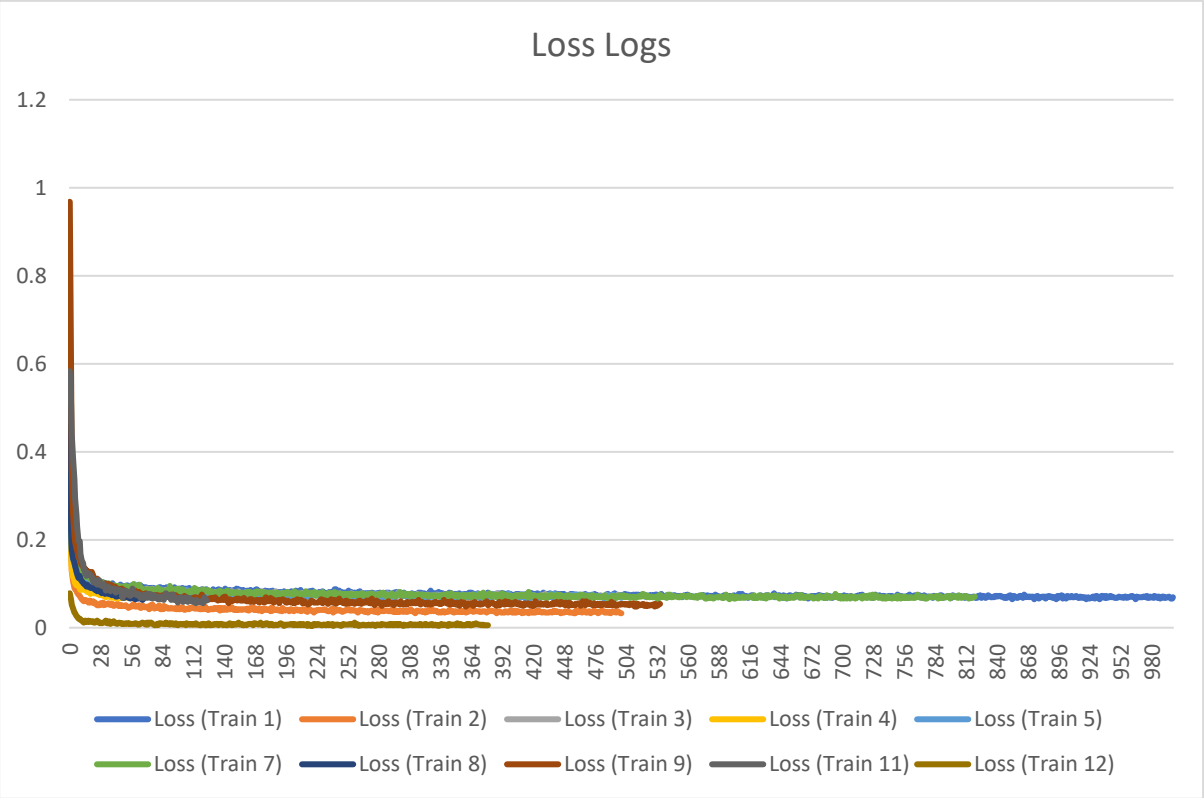
Train#	Loss	Blocks	Optimizer	Normalization	Regularization
1	MSE	BatchNorm2d	AdamW	Yes	No
2	MSE	BatchNorm2d	AdamW	Yes	No
3	MSE	BatchNorm2d	AdamW	Yes	No
4	MSE	BatchNorm2d	AdamW	Yes	No
5	MSE	GroupNorm + Dropout(0.1)	AdamW	Yes	Yes
6	MSE	GroupNorm + Dropout(0.1)	AdamW	Yes	Yes
7	MSE	GroupNorm + Dropout(0.2)	AdamW(weight_decay=0.01)	Yes	Yes
8	MSE	GroupNorm + Dropout(0.2)	AdamW(weight_decay=0.01)	Yes	Yes
9	MSE	GroupNorm + Dropout(0.1)	AdamW(weight_decay=0.01)	Yes	Yes
10	MSE	GroupNorm + Dropout(0.1)	AdamW(weight_decay=0.01)	Yes	Yes
11	MSE	GroupNorm + Dropout(0.1)	AdamW(weight_decay=0.01)	Yes	Yes
12	MSE	GroupNorm	AdamW(weight_decay=0.01)	Yes	No

Training logs (Section 3)

Train#	Data Augmentation	Cropping	Outcome	Comment
1	No	Yes	No convergence. Highly unstable	
2	No	Yes	No convergence. Highly unstable	
3	No	Yes	No convergence. Highly unstable	
4	No	Yes	Slight convergence	
5	Yes	Yes	Better convergence. But unstable	Regularization
6	Yes	Yes	Better convergence	Regularization + Annealing
7	Yes (Color Jitter)	Yes	Stable outcomes but low convergence	Heavier Regularization
8	Yes (Color Jitter)	Yes	decided that "grey blur" is the safest answer	Bigger Batch & Sampled T
9	Yes (Color Jitter)	Yes	Multiple colous but low convergence	Lighter Regularization & Bigger Batch
10	Yes (Color Jitter)	Yes	Multiple colous but low convergence	Higher T & Biggest Batch
11	Yes (Color Jitter)	Yes	Better convergence. Flowers shapes achieved.	
12	Yes (Color Jitter)	Yes	Sudden drop in Loss but stuck in local minima	Dropped Dropout. Added Val dataset. All images.

I found that the path to a stable model required significant trial and error as documented in my training logs. My early attempts with batch normalization and lower batch sizes resulted in highly unstable training and no convergence, which led me to transition toward group normalization and dropout for better regularization. I observed that increasing the batch size and pooling the entire dataset including the validation and test sets helped the model move past the "grey blur" phase seen in earlier trials. While I noticed that dropping dropout in my final run caused the loss to drop suddenly, it also caused the model to get stuck in a local minima. If I had more time, I would have explored the higher resolution of 128x128 more thoroughly as my trials showed it produced better flower shapes,

and I would have refined the learning rate warm-up to prevent the instabilities I encountered in the first few epochs.

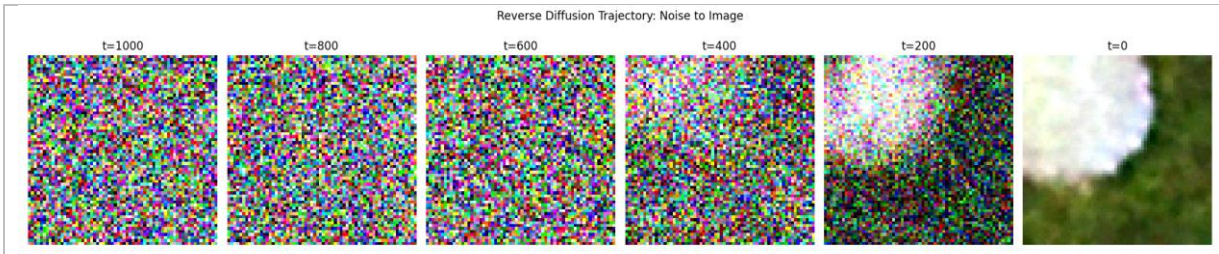


Future Improvements

To further refine the model, I would:

- Only if had more time to work with more epochs and edit data loader with transform function
- Experiment with Dropout layers in the bottleneck for enhanced regularization against diverse textures.
- Be allowed to fix the data loader script – specifically the center crop and resize transforms

Results


Reverse Diffusion Trajectory: Noise to Image
t=1000
t=800
t=600
t=400
t=200
t=0


Final Unconditional Samples